



UNIVALI

Universidade do Vale do Itajaí
Escola do Mar, Ciência e Tecnologia - EMCT
Ciência da Computação

ALGORITMO GENÉTICO PARA RESOLUÇÃO DO PROBLEMA DO CAIXEIRO VIAJANTE

Gustavo Henrique Stahl Müller
Luan Luiz Gambeta
Paulo Henrique Rohling

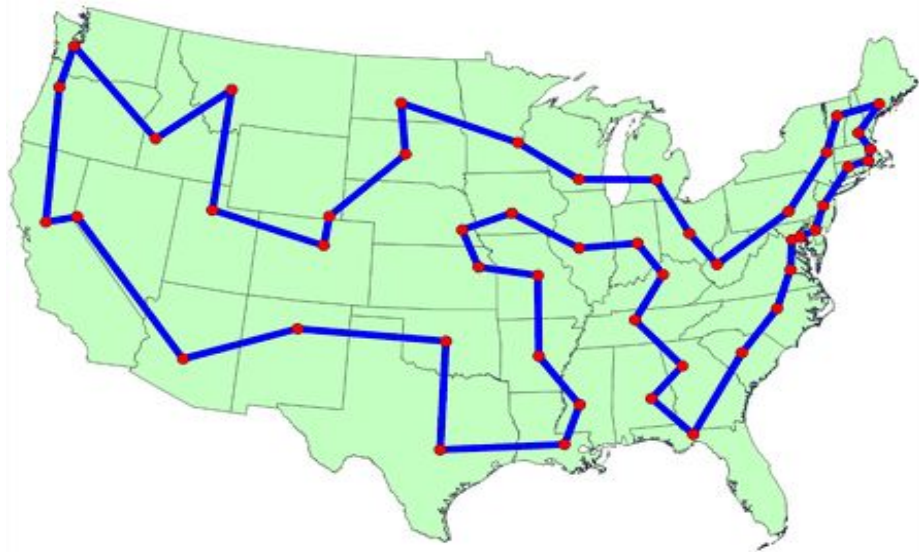


Contextualização

O **Problema do Caixeiro Viajante** é um problema de otimização que consiste em encontrar a **rota com o menor custo** entre um conjunto de cidades;

A rota candidata deve:

- **Iniciar e terminar** na mesma cidade;
- Passar em todas as cidade **uma única vez**;



Complexidade

A complexidade deste problema cresce em **proporções fatoriais**, uma vez que a quantidade de rotas que deve ser analisada é dada por:

$$\text{Rotas}(N) = (N - 1)!$$

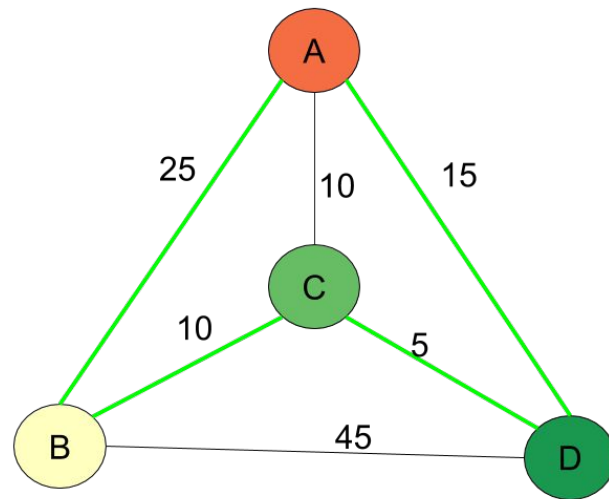
N	Rotas por segundo	Rotas(N)	Tempo necessário
5	250 milhões	24	Insignificante
10	110 milhões	362 880	0.003 segundo
15	71 milhões	87 bilhões	20 minutos
20	53 milhões	1.2×10^{17}	73 anos
25	42 milhões	6.2×10^{23}	470 milhões de anos

Fonte: <http://www.mat.ufrgs.br/~portosil/caixeiro.html>

Proposta

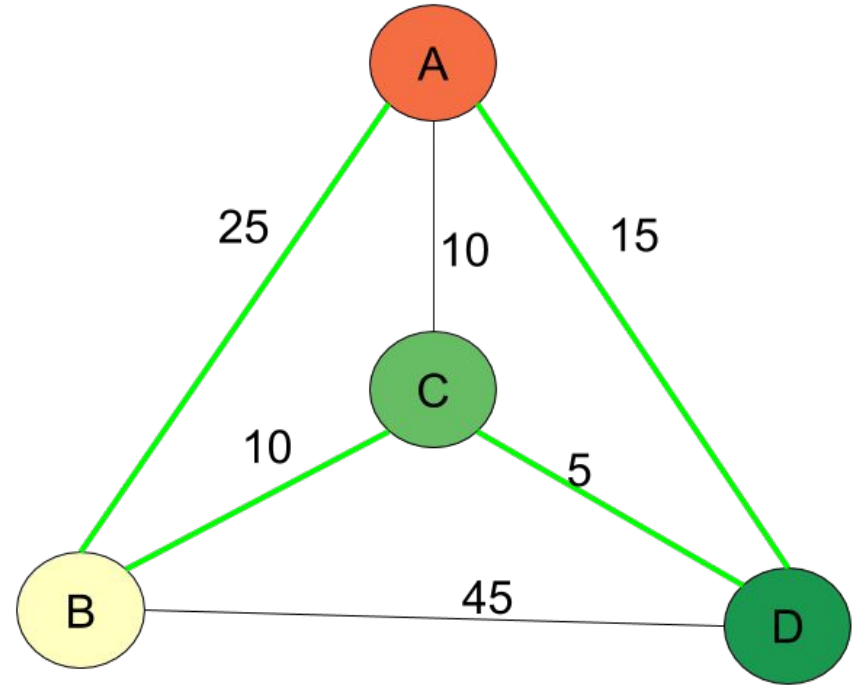
A imensa complexidade computacional **inviabiliza** a resolução através da **força bruta**. Logo, a proposta é utilizar um **Algoritmo Genético** para encontrar uma boa rota em um tempo aceitável;

- Estruturação do problema utilizando um **grafo**;
- Função de aptidão é o **custo da rota * -1**;
- Método de seleção por **Torneio de Três**;
- Cruzamento por **Swap em Pares**;



Estruturação

	A	B	C	D
A	-1	25	10	15
B	25	-1	10	45
C	10	10	-1	5
D	15	45	5	-1



Representação

A representação do cromossomo **não inclui o último gene**, pois este será sempre igual ao primeiro gene do cromossomo. Exemplo: [**A, B, C, D**]

Cálculo de Aptidão

Os indivíduos são classificados através do **comprimento total** da rota. O indivíduo acima teria *fitness* = **-55**, uma vez que:

$$fitness = custoDaRota * -1$$

$$fitness = (25 + 10 + 5 + 15) * -1 = \mathbf{-55}$$

	A	B	C	D
A	-1	25	10	15
B	25	-1	10	45
C	10	10	-1	5
D	15	45	5	-1

Representação

A representação do cromossomo **não inclui o último gene**, pois este será sempre igual ao primeiro gene do cromossomo. Exemplo: [**A**, **B**, **C**, **D**]

Cálculo de Aptidão

Os indivíduos são classificados através do **comprimento total** da rota. O indivíduo acima teria *fitness* = **-55**, uma vez que:

$$fitness = custoDaRota * -1$$

$$fitness = (25 + 10 + 5 + \textcircled{15}) * \textcircled{-1} = \textbf{-55}$$

	A	B	C	D
A	-1	25	10	15
B	25	-1	10	45
C	10	10	-1	5
D	15	45	5	-1

Seleção

O método utilizado para selecionar os indivíduos de acordo com a sua aptidão é o **Torneio de Três**, onde três indivíduos aleatórios têm sua aptidão comparada, resultando na seleção do mais apto.

$$[A, B, C, D] = -55$$

$$[A, B, D, C] = -85$$

$$[A, C, B, D] = -80$$

- Descarta rapidamente indivíduos **pouco aptos**;
- Tende a diminuir a **diversidade** da população;
- Converge rapidamente para um **máximo local**.

	A	B	C	D
A	-1	25	10	15
B	25	-1	10	45
C	10	10	-1	5
D	15	45	5	-1

Algoritmo de Seleção

algoritmo TorneioDeTrês(populaçãoAtual)

vencedoresDoTorneio = []

para cada indivíduo **em** populaçãoAtual **faça**

se *RandomizarChance()* <= TAXA_DE_CRUZAMENTO **então**

 indivíduo1 = *PegarIndivíduoAleatório*(populaçãoAtual)

 indivíduo2 = *PegarIndivíduoAleatório*(populaçãoAtual)

 indivíduo3 = *PegarIndivíduoAleatório*(populaçãoAtual)

 vencedor = *MaiorFitness*(indivíduo1, indivíduo2, indivíduo3);

 vencedoresDoTorneio += vencedor

fim-se

fim-para

retorna vencedoresDoTorneio

fim-algoritmo

Cruzamento

O cruzamento de dois indivíduos é feito através da técnica de **Swap em Pares**, onde:

- É sorteada a posição de um gene do cromossomo A;
- Verifica-se qual a posição deste gene no cromossomo B;
- É feito um swap entre a posição A e a posição B tanto no cromossomo A quanto no cromossomo B.

0	1	2	3	0	1	2	3	0	1	2	3
[A	B	C	D]	[A	B	C	D]	[A	C	B	D]
[D	C	B	A]	[D	C	B	A]	[D	B	C	A]

Algoritmo de Cruzamento

algoritmo SwapEmPares(indivíduo1, indivíduo2)

posiçãoIndivíduo1 = *RandomizarPosição()*

gene1 = indivíduo1.genes[posiçãoIndivíduo1]

posiçãoIndivíduo2 = *PegarPosiçãoDoGene*(gene1)

Swap(indivíduo1.genes[posiçãoIndivíduo1], indivíduo1.genes[posiçãoIndivíduo2])

Swap(indivíduo2.genes[posiçãoIndivíduo1], indivíduo2.genes[posiçãoIndivíduo2])

fim-algoritmo

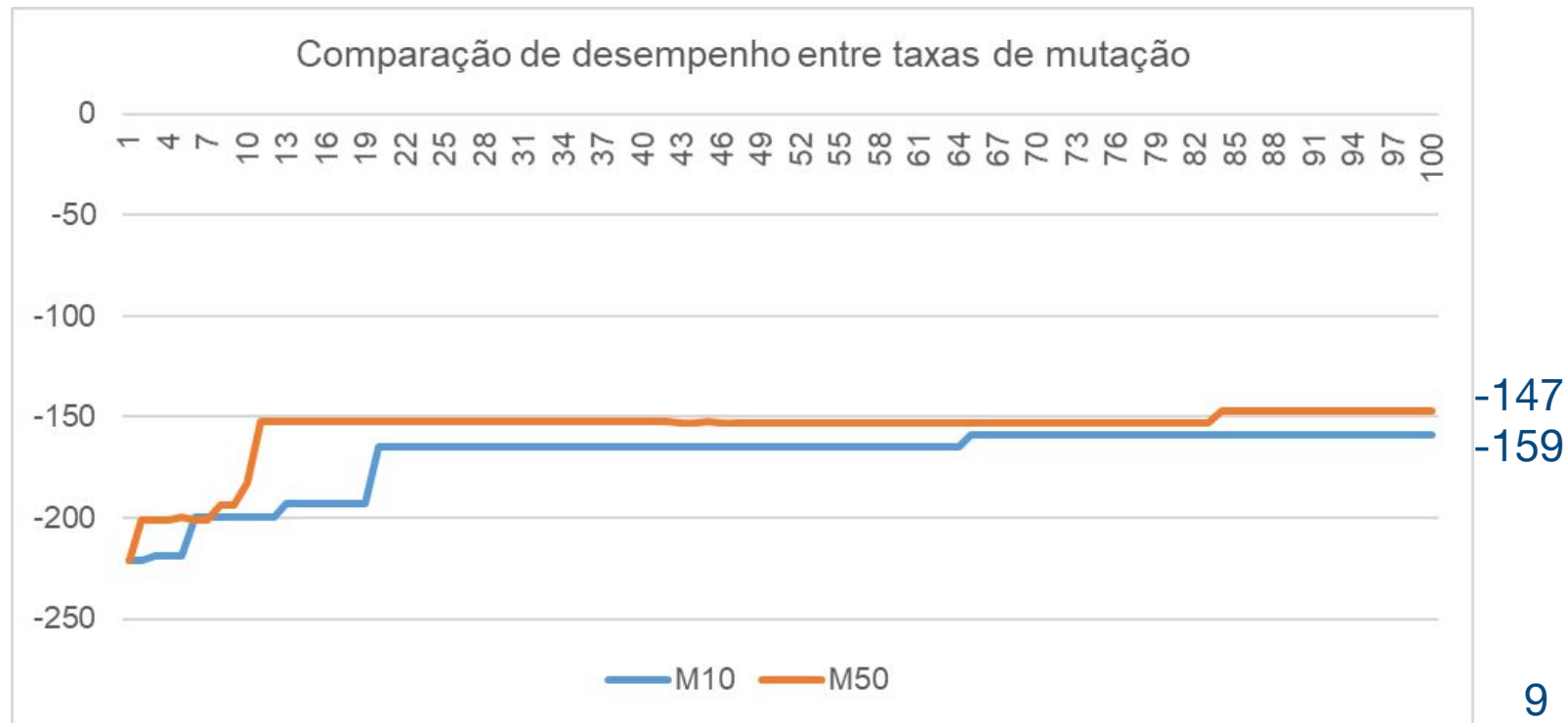
Mutação

O método de mutação utilizado é o **Swap Simples**, onde dois genes são simplesmente trocados.

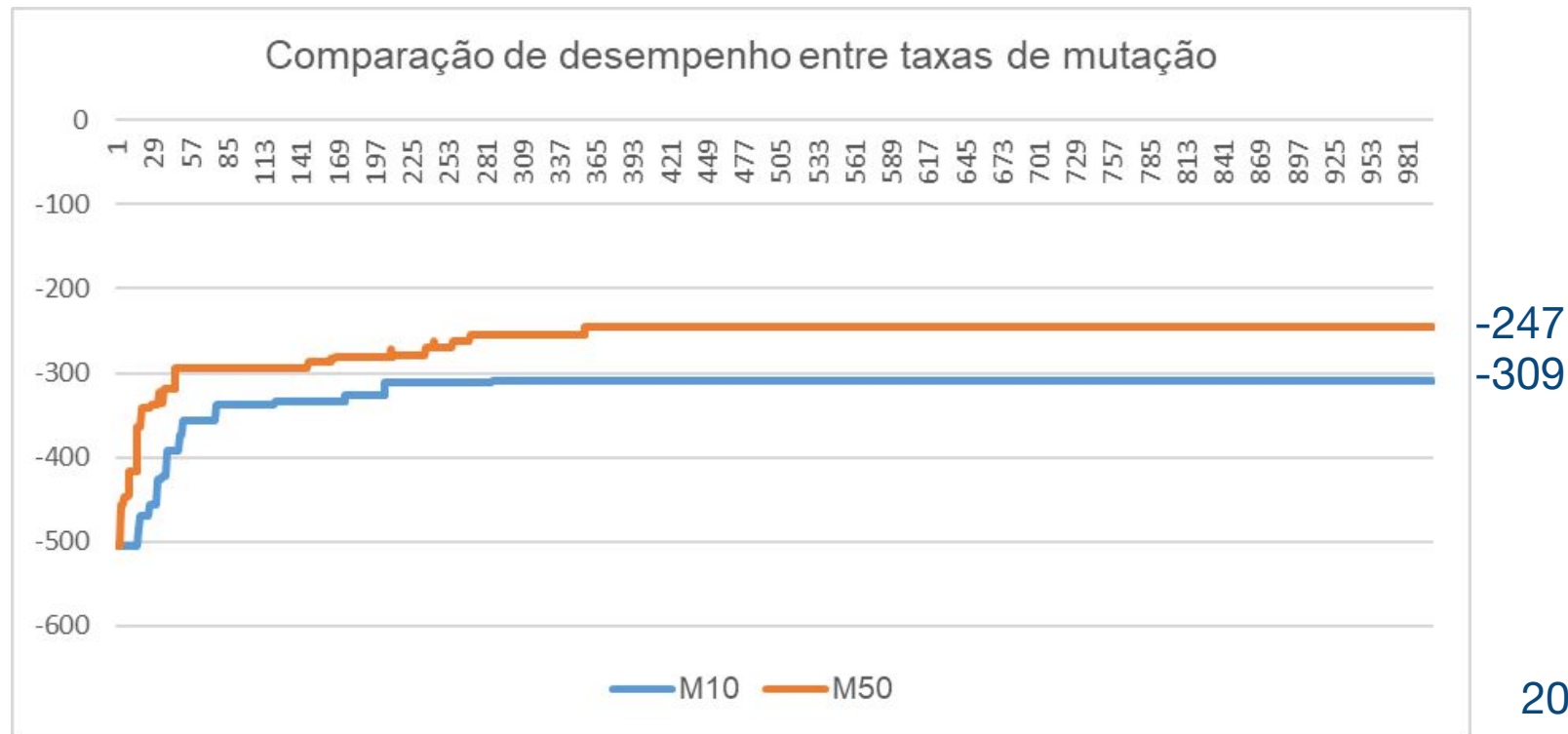
0	1	2	3	0	1	2	3										
[	A	,	B	,	C	,	D	]	[	B	,	A	,	C	,	D	]

- Extremamente importante manter a **taxa de mutação alta** para contrabalancear o elitismo do **Torneio de Três**.

Mutação



Mutação





UNIVALI

Universidade do Vale do Itajaí
Escola do Mar, Ciência e Tecnologia - EMCT
Ciência da Computação

ALGORITMO GENÉTICO PARA RESOLUÇÃO DO PROBLEMA DO CAIXEIRO VIAJANTE

Gustavo Henrique Stahl Müller
Luan Luiz Gambeta
Paulo Henrique Rohling

